Deep Dive: Porting my eBPF anti-ransomware to Rust



AUTHOR
JustThinkingHard
Security engineer | Low-level enjoyer

Disclaimer: This project is the Rust version of my eBPF project in C explained here : *anti-ransomware post*.

Writing the C version of the anti-ransomware, I figured that the problem with writing it in C was that I could literally do anything I wanted, meaning that I could also make a lot of mistakes (race conditions, memory allocation errors...). Writing in C implies that the author has a deep level of knowledge about what he is trying to do and how to do it, because the whole safety of his code is based on himself. However, since I am human (and you hopefully are too), it meant that I would obviously make a mistake at any point in this program.

This whole thinking process made me want to rewrite the whole project into Rust, where there are lots of rules inherent to the programming language. Rust would add a layer of security on top of the author, by checking its work through a set of rules that are just not present in C. However, even though Rust seems

a really good match for this low-level project, there is a down side: the use of `unsafe`. Rust has a lot of rules, but obviously, if I am trying to work with raw memory or pointers, Rust can not be 100% sure that it is safe, since it can't know for sure the outcome of what I am doing. For this exact point, Rust hands me the `unsafe` keyword, used to tell it that I am sure of what I am writing, and that it doesn't have to check it. This means that, on those little portions of code, we get back to writing code whose only safeguard is me, but since it is mandatory to work on low level side of the project, we accept it.

So in this post, I wanted to review what changed between the two versions, and why.

Rust verifier and e-BPF verifier

On a C project with e-BPF, you would code your project, hand it to the e-BPF verifier, and it would analyse it to tell you if it is assumed "safe to run". What happens if you already have a verifier that is trying to do the same work before ? It makes it easier ?

Well, no.

Because the Rust verifier and the e-BPF verifier are not doing the exact same job and, on certain occasions, it can lead to an error :

Back when I was trying to log the command name of the process in the e-BPF, I added a typical `core::str::from_utf8(...)` to retrieve characters.

```
R1 bitwise operator &= on pointer prohibited
```

Rust did not report any mistake, but e-BPF did, why ? Well, `from_utf8` is actually trying to validate the value. The compiled form of that validation does a bitwise operation, and the verifier traces one of its operands back to a pointer. Bitwise arithmetic on a pointer is a no-go for the verifier, because you could use it to reach arbitrary memory. Thus, the e-BPF verifier is denying my program for safety reasons, forcing me to use `from_utf8_unchecked` but if we look at its signature :

```
pub const unsafe fn from_utf8_unchecked(v: &[u8]) ->
```

It is an `unsafe` function !

Obviously, this example is not really dangerous, since it was only used for debug purposes and then wiped from the code, but it shows that on some occasions, having two verifiers does not mean that they will accept the same code.

Rust error management creating more errors

One of the best parts of using Rust is being able to have a proper error management. You can write your functions and then either return the result of that function, or the error, and Rust will know which is which. This is possible with the keyword `Result` .

```
fn foo() -> Result<u64, i32> {
```

On this example, we can see that the `foo` function is either returning a `u64`, for example a `i_node`, or a `i32`, typically an error code like `-EPERM` (`-1`). When we call the `foo` function, we can choose to handle this error directly or, and this is the beauty of it, we can forward it (the equivalent of doing a `return` of the error code) with `?`.

So if we do:

```
let i_node = foo()?;
```

`i_node` will be a `u64` value, because if `foo` returns an error, it will be forwarded.

Now, why is it a problem with e-BPF ?

Well, let's take this function :

```
pub unsafe fn bpf_probe_read_kernel<T>(src: *const T
```

It is used to read bytes. If it works, it will return a typed `T` value. If it fails, it will return an `i32` value, but it says in the documentation that it will always be a negative value. So let's assume we use this function in a LSM hook:

```
fn LSM_hook(ctx: LSMContext) -> Result<i32, i32> {
    ...
    let ia_valid: u32 = unsafe { bpf_probe_read_kerr
    ...
```

```
}
```

So we retrieve `ia_valid`, a `u32` value, with `bpf_probe_read_kernel`, but what happens if it fails ? Since we put `?` at the end, it means that we forward the error.

Everything is okay, right ? Well, no again.

```
At program exit the register R0 has unknown scalar v
```

But why ? We know that `bpf_probe_read_kernel` can only return negative `i32` value, which is what we want ? Yes but no. There is nothing preventing `bpf_probe_read_kernel` from returning a negative value less than `-4095`, even if it is unlikely, but the problem is not there. The e-BPF verifier has no proof that `bpf_probe_read_kernel` is returning only negative values !

Us, humans, we have agreed that this function shall return only negative values, but for a machine, `i32` means `"a value from -2 147 483 648 to 2 147 483 647"`. Because we forwarded it without checking it, the e-BPF verifier is assuming that the return value can be dangerous, and thus denying it.

To fix this, we use `match` to check the error, and making sure that it is allowed :

```
    let ia_valid = match unsafe { bpf_probe_read_ker
        Ok(i) => i,
        Err(_) => return Err(-1),
    };
```

Using Rust to build this eBPF program helped in many ways, but it did not remove the need to understand what I was doing. It kept me on the safe path for most of the road, exactly like the eBPF verifier does, but neither of them thinks for you. In the end, Rust didn’t remove the need to understand the kernel, it just moved where I had to be careful.